



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Introduction to Data Science

Mid-Term Project Report

Spring 2025-2026

Section: F

Group:10

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: Afnan Bin Islam Nahin ID:22-49350-3	Dataset selection, data understanding, preprocessing, descriptive statistics, report writing
2.	Name: Aman Ullah ID:22-49345-3	Data cleaning, outlier analysis, normalization, visualization, output collection
3.	Name: ID:	
4.	Name: ID:	

Dataset Description

The dataset used in this project is the IBM HR Analytics Employee Attrition dataset. This dataset contains employee-related information and is used to analyze the factors that may influence employee attrition.

The dataset consists of 1470 rows and 35 columns. Each row represents one employee, and each column represents a specific attribute of that employee. The dataset contains both numerical and categorical features, which makes it suitable for data preprocessing and descriptive statistical analysis.

Some important numerical attributes in the dataset are Age, MonthlyIncome, DailyRate, DistanceFromHome, TotalWorkingYears, and YearsAtCompany. Important categorical attributes include Attrition, Gender, Department, JobRole, MaritalStatus, BusinessTravel, and OverTime.

The target variable of this dataset is Attrition, which is categorical in nature. It has two classes:

- **Yes** = the employee has left the company
- **No** = the employee is still working in the company

This dataset is suitable for the project because it contains both categorical and numerical variables, has a clearly defined class label, and supports different preprocessing tasks such as missing value checking, duplicate detection, outlier handling, normalization, filtering, class comparison, and variability analysis.

Project Implementation Detail

Title of the task1: Load Dataset and Preview

Description of the task1: This task loads the IBM HR Attrition dataset into R and displays the first few rows to get an initial view of the data.

Code for task1

```
hr_raw <- read.csv("WA_Fn-UseC_-HR-Employee-Attrition.csv", stringsAsFactors = FALSE)
head(hr_raw[, c("Age", "Attrition", "Department", "JobRole", "MonthlyIncome", "OverTime")])
```

Sample output of your code for task1

```
> head(hr_raw[, c("Age", "Attrition", "Department", "JobRole", "MonthlyIncome", "OverTime")])
  Age Attrition Department JobRole MonthlyIncome OverTime
1  41        Yes      Sales Sales Executive      5993      Yes
2  49         No Research & Development Research Scientist      5130      No
3  37        Yes Research & Development Laboratory Technician      2090      Yes
4  33         No Research & Development Research Scientist      2909      Yes
5  27         No Research & Development Laboratory Technician      3468      No
6  32         No Research & Development Laboratory Technician      3068      No
> |
```

Code description of task1: The `read.csv()` function is used to import the dataset, and `head()` is used to display the first six observations for quick inspection.

Title of the task2: Dataset Shape

Description of the task2: This task checks the size of the dataset by showing the total number of rows and columns.

Code for task2

```
dim(hr_raw)
```

Sample output of your code for task2

```
> dim(hr_raw)
[1] 1470  35
> |
```

Code description of task2: The `dim()` function returns the number of observations and the number of attributes in the dataset.

Title of the task3: Feature Names

Description of the task3: This task displays all column names in the dataset.

Code for task3

```
names(hr_raw)
```

Sample output of your code for task3

```
> names(hr_raw)
[1] "Age" "Attrition" "BusinessTravel" "DailyRate" "Department"
[6] "DistanceFromHome" "Education" "EducationField" "EmployeeCount" "EmployeeNumber"
[11] "EnvironmentSatisfaction" "Gender" "HourlyRate" "JobInvolvement" "JobLevel"
[16] "JobRole" "JobSatisfaction" "MaritalStatus" "MonthlyIncome" "MonthlyRate"
[21] "NumCompaniesWorked" "Over18" "OverTime" "PercentSalaryHike" "PerformanceRating"
[26] "RelationshipSatisfaction" "StandardHours" "StockOptionLevel" "TotalWorkingYears" "TrainingTimesLastYear"
[31] "WorkLifeBalance" "YearsAtCompany" "YearsInCurrentRole" "YearsSinceLastPromotion" "YearsWithCurrManager"
```

Code description of task3: The `names()` function is used to print the names of all columns.

Title of the task4: Data Structure

Description of the task4: This task examines the structure of the dataset and identifies the data type of each feature.

Code for task4

```
str(hr_raw)
```

Sample output of your code for task4

```
> str(hr_raw)
'data.frame': 1470 obs. of 35 variables:
 $ Age : int 41 49 37 33 27 32 59 30 38 36 ...
 $ Attrition : chr "Yes" "No" "Yes" "No" ...
 $ BusinessTravel : chr "Travel_Rarely" "Travel_Frequently" "Travel_Rarely" "Travel_Frequently" ...
 $ DailyRate : int 1102 279 1373 1392 591 1005 1324 1358 216 1299 ...
 $ Department : chr "Sales" "Research & Development" "Research & Development" "Research & Development" ...
 $ DistanceFromHome : int 1 8 2 3 2 2 3 24 23 27 ...
 $ Education : int 2 1 2 4 1 2 3 1 3 3 ...
 $ EducationField : chr "Life Sciences" "Life Sciences" "Other" "Life Sciences" ...
 $ EmployeeCount : int 1 1 1 1 1 1 1 1 1 1 ...
 $ EmployeeNumber : int 1 2 4 5 7 8 10 11 12 13 ...
 $ EnvironmentSatisfaction : int 2 3 4 4 1 4 3 4 4 3 ...
 $ Gender : chr "Female" "Male" "Male" "Female" ...
 $ HourlyRate : int 94 61 92 56 40 79 81 67 44 94 ...
 $ JobInvolvement : int 3 2 2 3 3 3 4 3 2 3 ...
 $ JobLevel : int 2 2 1 1 1 1 1 1 3 2 ...
 $ JobRole : chr "Sales Executive" "Research Scientist" "Laboratory Technician" "Research Scientist" ...
 $ JobSatisfaction : int 4 2 3 3 2 4 1 3 3 3 ...
 $ MaritalStatus : chr "Single" "Married" "Single" "Married" ...
 $ MonthlyIncome : int 5993 5130 2090 2909 3468 3068 2670 2693 9526 5237 ...
 $ MonthlyRate : int 19479 24907 2396 23159 16632 11864 9964 13335 8787 16577 ...
 $ NumCompaniesWorked : int 8 1 6 1 9 0 4 1 0 6 ...
 $ Over18 : chr "Yes" "No" "Yes" "No" ...
 $ OverTime : chr "Yes" "No" "Yes" "Yes" ...
 $ PercentSalaryHike : int 11 23 15 11 12 13 20 22 21 13 ...
 $ PerformanceRating : int 3 4 3 3 3 3 4 4 4 3 ...
 $ RelationshipSatisfaction : int 1 4 2 3 4 3 1 2 2 2 ...
 $ StandardHours : int 80 80 80 80 80 80 80 80 80 80 ...
 $ StockOptionLevel : int 0 1 0 0 1 0 3 1 0 2 ...
 $ TotalWorkingYears : int 8 10 7 8 6 8 12 1 10 17 ...
 $ TrainingTimesLastYear : int 0 3 3 3 3 2 2 2 2 3 ...
 $ WorkLifeBalance : int 1 3 3 3 3 2 2 3 3 2 ...
 $ YearsAtCompany : int 6 10 0 8 2 7 1 1 9 7 ...
 $ YearsInCurrentRole : int 4 7 0 7 2 7 0 0 7 7 ...
 $ YearsSinceLastPromotion : int 0 1 0 3 2 3 0 0 1 7 ...
 $ YearsWithCurrManager : int 5 7 0 0 2 6 0 0 8 7 ...
```

Code description of task4: The `str()` function provides a compact summary of the dataset structure and variable types.

Title of the task5: Identify Numeric and Categorical Columns

Description of the task5: This task separates numerical and categorical variables for further preprocessing and analysis.

Code for task5

```
numeric_cols <- names(hr_raw)[sapply(hr_raw, is.numeric)]
categorical_cols <- names(hr_raw)[sapply(hr_raw, is.character)]

length(numeric_cols)

length(categorical_cols)

numeric_cols

categorical_cols
```

Sample output of your code for task5

```
> length(numeric_cols)
[1] 26
> length(categorical_cols)
[1] 9
> numeric_cols
 [1] "Age" "DailyRate" "DistanceFromHome" "Education" "EmployeeCount"
 [6] "EmployeeNumber" "EnvironmentSatisfaction" "HourlyRate" "JobInvolvement" "JobLevel"
[11] "JobSatisfaction" "MonthlyIncome" "MonthlyRate" "NumCompaniesWorked" "PercentSalaryHike"
[16] "PerformanceRating" "RelationshipSatisfaction" "StandardHours" "StockOptionLevel" "TotalWorkingYears"
[21] "TrainingTimesLastYear" "WorkLifeBalance" "YearsAtCompany" "YearsInCurrentRole" "YearsSinceLastPromotion"
[26] "YearsWithCurrManager"
> categorical_cols
 [1] "Attrition" "BusinessTravel" "Department" "EducationField" "Gender" "JobRole" "MaritalStatus" "Over18"
 [9] "OverTime"
```

Code description of task5: The `sapply()` function checks each column type and stores numeric and categorical column names separately.

Title of the task6: Dataset Summary Table

Description of the task6: This task creates a short summary table containing dataset name, number of rows, number of columns, number of numeric columns, number of categorical columns, and target variable.

Code for task6

```
ds_summary <- data.frame(
  Dataset = "IBM HR Analytics Employee Attrition",
  Rows = nrow(hr_raw),
  Columns = ncol(hr_raw),
  Numeric_Columns = length(numeric_cols),
  Categorical_Columns = length(categorical_cols),
  Target = "Attrition (Yes/No)",
  stringsAsFactors = FALSE
)
ds_summary
```

Sample output of your code for task6

```
+ )
> ds_summary
  Dataset Rows Columns Numeric_Columns Categorical_Columns Target
1 IBM HR Analytics Employee Attrition 1470      35          26      9 Attrition (Yes/No)
```

Code description of task6: A `data.frame()` is created manually to present a compact overview of the dataset.

Title of the task7: Attrition Class Distribution

Description of the task7: This task checks the distribution of the target variable Attrition and shows the class imbalance in the dataset.

Code for task7

```
attr_table <- table(hr_raw$Attrition)
attr_percent <- round(prop.table(attr_table) * 100, 2)
attr_table
attr_percent
```

Sample output of your code for task7

```
> attr_table
  No  Yes
1233 237
> attr_percent
  No  Yes
83.88 16.12
>
```

Code description of task7: The `table()` function counts the occurrences of each class, and `prop.table()` converts them into percentages.

Title of the task8: Summary of Key Numerical Features

Description of the task8: This task provides basic descriptive statistics for important numerical features.

Code for task8

```
summary(hr_raw[,
c("Age", "MonthlyIncome", "TotalWorkingYears", "DistanceFromHome")])
```

Sample output of your code for task8

```
> summary(hr_raw[, c("Age", "MonthlyIncome", "TotalWorkingYears", "DistanceFromHome")])
   Age      MonthlyIncome    TotalWorkingYears DistanceFromHome
Min.   :18.00   Min.    : 1009   Min.    : 0.00   Min.    : 1.000
1st Qu.:30.00   1st Qu.: 2911   1st Qu.: 6.00   1st Qu.: 2.000
Median :36.00   Median : 4919   Median :10.00   Median : 7.000
Mean   :36.92   Mean    : 6503   Mean    :11.28   Mean    : 9.193
3rd Qu.:43.00   3rd Qu.: 8379   3rd Qu.:15.00   3rd Qu.:14.000
Max.   :60.00   Max.    :19999   Max.    :40.00   Max.    :29.000
>
```

Code description of task8: The `summary()` function displays minimum, quartiles, median, mean, and maximum values.

Title of the task9: Missing Values Check

Description of the task9: This task checks whether the dataset contains any missing values.

Code for task9

```
na_col <- colSums(is.na(hr_raw))
na_col
sum(na_col)
```

Sample output of your code for task9

```
> na_col <- colSums(is.na(hr_raw))
> na_col
      Age      Attrition      BusinessTravel      DailyRate      Department
      0         0         0         0         0
DistanceFromHome      Education      EducationField      EmployeeCount      EmployeeNumber
      0         0         0         0         0
EnvironmentSatisfaction      Gender      HourlyRate      JobInvolvement      JobLevel
      0         0         0         0         0
      JobRole      JobSatisfaction      MaritalStatus      MonthlyIncome      MonthlyRate
      0         0         0         0         0
NumCompaniesWorked      Over18      OverTime      PercentSalaryHike      PerformanceRating
      0         0         0         0         0
RelationshipSatisfaction      StandardHours      StockOptionLevel      TotalWorkingYears      TrainingTimesLastYear
      0         0         0         0         0
      WorkLifeBalance      YearsAtCompany      YearsInCurrentRole      YearsSinceLastPromotion      YearsWithCurrManager
      0         0         0         0         0
> sum(na_col)
[1] 0
> |
```

Code description of task9: The `is.na()` function identifies missing values, and `colSums()` counts them column-wise.

Title of the task10: Blank String to NA Check

Description of the task10: This task converts blank strings in categorical columns into missing values and checks the dataset again.

Code for task10

```
hr_step <- hr_raw
for (col in categorical_cols) {
  hr_step[[col]][trimws(hr_step[[col]]) == ""] <- NA
}
na_col2 <- colSums(is.na(hr_step))
na_col2
sum(na_col2)
```

Sample output of your code for task10

```

> na_col2 <- colSums(is.na(hr_step))
> na_col2
      Age      Attrition      BusinessTravel      DailyRate      Department
      0         0         0         0         0
DistanceFromHome      Education      EducationField      EmployeeCount      EmployeeNumber
      0         0         0         0         0
EnvironmentSatisfaction      Gender      HourlyRate      JobInvolvement      JobLevel
      0         0         0         0         0
      JobRole      JobSatisfaction      MaritalStatus      MonthlyIncome      MonthlyRate
      0         0         0         0         0
NumCompaniesWorked      Over18      OverTime      PercentSalaryHike      PerformanceRating
      0         0         0         0         0
RelationshipSatisfaction      StandardHours      StockOptionLevel      TotalWorkingYears      TrainingTimesLastYear
      0         0         0         0         0
      WorkLifeBalance      YearsAtCompany      YearsInCurrentRole      YearsSinceLastPromotion      YearsWithCurrManager
      0         0         0         0         0
> sum(na_col2)
[1] 0
>

```

Code description of task10: `trimws()` removes extra spaces, and blank strings are replaced with NA.

Title of the task11: Missing Value Handling

Description of the task11: This task applies median imputation for numerical variables and mode imputation for categorical variables.

Code for task11

```

get_mode <- function(x) {
  ux <- unique(x[!is.na(x)])
  ux[which.max(tabulate(match(x, ux)))]
}

hr_imputed <- hr_step

na_before <- colSums(is.na(hr_imputed))

for (col in numeric_cols) {
  hr_imputed[[col]][is.na(hr_imputed[[col]])] <- median(hr_imputed[[col]], na.rm = TRUE)
}

for (col in categorical_cols) {
  hr_imputed[[col]][is.na(hr_imputed[[col]])] <- get_mode(hr_imputed[[col]])
}

na_after <- colSums(is.na(hr_imputed))

```

na_before

na_after

Sample output of your code for task11

```
> for (col in categorical_cols) {
+   hr_imputed[[col]][is.na(hr_imputed[[col]])] <- get_mode(hr_imputed[[col]])
+ }
>
> na_before
      Age      Attrition      BusinessTravel      DailyRate      Department
      0           0           0           0           0
DistanceFromHome      Education      EducationField      EmployeeCount      EmployeeNumber
      0           0           0           0           0
EnvironmentSatisfaction      Gender      HourlyRate      JobInvolvement      JobLevel
      0           0           0           0           0
      JobRole      JobSatisfaction      MaritalStatus      MonthlyIncome      MonthlyRate
      0           0           0           0           0
NumCompaniesWorked      Over18      OverTime      PercentSalaryHike      PerformanceRating
      0           0           0           0           0
RelationshipSatisfaction      StandardHours      StockOptionLevel      TotalWorkingYears      TrainingTimesLastYear
      0           0           0           0           0
      WorkLifeBalance      YearsAtCompany      YearsInCurrentRole      YearsSinceLastPromotion      YearsWithCurrManager
      0           0           0           0           0
> na_after <- colSums(is.na(hr_imputed))
> na_after
      Age      Attrition      BusinessTravel      DailyRate      Department
      0           0           0           0           0
DistanceFromHome      Education      EducationField      EmployeeCount      EmployeeNumber
      0           0           0           0           0
EnvironmentSatisfaction      Gender      HourlyRate      JobInvolvement      JobLevel
      0           0           0           0           0
      JobRole      JobSatisfaction      MaritalStatus      MonthlyIncome      MonthlyRate
      0           0           0           0           0
NumCompaniesWorked      Over18      OverTime      PercentSalaryHike      PerformanceRating
      0           0           0           0           0
RelationshipSatisfaction      StandardHours      StockOptionLevel      TotalWorkingYears      TrainingTimesLastYear
      0           0           0           0           0
      WorkLifeBalance      YearsAtCompany      YearsInCurrentRole      YearsSinceLastPromotion      YearsWithCurrManager
      0           0           0           0           0
> |
```

Code description of task11: A custom mode function is used for categorical columns, while median() is used for numerical columns to fill missing values.

Title of the task12: Recommended Columns to Remove

Description of the task12: This task identifies columns that are constant or act like identifiers and are not useful for analysis.

Code for task12

```
remove_tbl <- data.frame(
  Column = c("EmployeeCount", "EmployeeNumber", "Over18", "StandardHours"),
  Reason = c(
    "Constant column",
    "Identifier-like column",
    "Constant column",
    "Constant column"
  ),
)
```

```

UniqueValues = c(
  length(unique(hr_imputed$EmployeeCount)),
  length(unique(hr_imputed$EmployeeNumber)),
  length(unique(hr_imputed$Over18)),
  length(unique(hr_imputed$StandardHours))
),
stringsAsFactors = FALSE
)

```

```
remove_tbl
```

Sample output of your code for task12

```

+ )
> remove_tbl
  Column          Reason UniqueValues
1 EmployeeCount  Constant column      1
2 EmployeeNumber Identifier-like column 1470
3 Over18         Constant column      1
4 StandardHours  Constant column      1
>

```

Code description of task12: A small table is created to explain why selected columns should be removed.

Title of the task13: Remove Redundant Columns

Description of the task13: This task removes unnecessary columns and confirms the updated dataset size.

Code for task13

```

remove_cols <- c("EmployeeCount", "EmployeeNumber", "Over18", "StandardHours")
hr <- hr_imputed[, !(names(hr_imputed) %in% remove_cols)]
dim(hr)
setdiff(remove_cols, names(hr))

```

Sample output of your code for task13

```

> remove_cols <- c("EmployeeCount", "EmployeeNumber", "Over18", "StandardHours")
> hr <- hr_imputed[, !(names(hr_imputed) %in% remove_cols)]
> dim(hr)
[1] 1470 31
> setdiff(remove_cols, names(hr))
[1] "EmployeeCount" "EmployeeNumber" "Over18" "StandardHours"
>

```

Code description of task13: Columns are removed using logical indexing with %in%.

Title of the task14: Duplicate Rows Check and Removal

Description of the task14: This task checks whether any duplicate rows exist and removes them if found.

Code for task14

```

dup_before <- sum(duplicated(hr))

rows_before <- nrow(hr)

hr <- hr[!duplicated(hr), ]

rows_after <- nrow(hr)

dup_before

c(Before = rows_before, After = rows_after)

```

Sample output of your code for task14

```

> dup_before <- sum(duplicated(hr))
> rows_before <- nrow(hr)
> hr <- hr[!duplicated(hr), ]
> rows_after <- nrow(hr)
> dup_before
[1] 0
> c(Before = rows_before, After = rows_after)
Before After
1470 1470
>

```

Code description of task14: The duplicated() function identifies repeated rows, and only unique rows are kept.

Title of the task15: Convert Character Columns to Factor

Description of the task15: This task converts categorical text columns into factor type for grouped analysis.

Code for task15

```
char_cols <- names(hr)[sapply(hr, is.character)]
for (col in char_cols) {
  hr[[col]] <- as.factor(hr[[col]])
}
char_cols
sapply(hr[char_cols], class)
lapply(hr[, c("Attrition","OverTime","Department")], levels)
```

Sample output of your code for task15

```
+ }
> char_cols
[1] "Attrition"      "BusinessTravel" "Department"      "EducationField" "Gender"          "JobRole"          "MaritalStatus"  "OverTime"
> sapply(hr[char_cols], class)
Attrition BusinessTravel Department EducationField Gender JobRole MaritalStatus OverTime
"factor"    "factor"    "factor"    "factor"    "factor"    "factor"    "factor"    "factor"
> lapply(hr[, c("Attrition","OverTime","Department")], levels)
$Attrition
[1] "No" "Yes"

$OverTime
[1] "No" "Yes"

$Department
[1] "Human Resources"      "Research & Development" "Sales"
```

Code description of task15: The `as.factor()` function is applied to all character columns.

Title of the task16: Selected Variables Table

Description of the task16: This task documents the main variables used in the analysis and their roles.

Code for task16

```
analysis_vars <- data.frame(
  Variable = c("Attrition","Age","Gender","Department","JobRole","OverTime",
    "MonthlyIncome","DailyRate","DistanceFromHome","TotalWorkingYears",
    "YearsAtCompany","JobLevel","JobSatisfaction","EnvironmentSatisfaction",
    "WorkLifeBalance","YearsSinceLastPromotion","TrainingTimesLastYear"),
```

```

Role      =      c("Target","Demographic","Demographic","Work context","Work
context","Workload",
      "Compensation","Compensation","Commute","Experience",
      "Tenure","Seniority","Satisfaction","Satisfaction",
      "Satisfaction","Career growth","Training"),
stringsAsFactors = FALSE
)

```

analysis_vars

Sample output of your code for task16

```

+ )
> analysis_vars

```

	Variable	Role
1	Attrition	Target
2	Age	Demographic
3	Gender	Demographic
4	Department	Work context
5	JobRole	Work context
6	OverTime	Workload
7	MonthlyIncome	Compensation
8	DailyRate	Compensation
9	DistanceFromHome	Commute
10	TotalWorkingYears	Experience
11	YearsAtCompany	Tenure
12	JobLevel	Seniority
13	JobSatisfaction	Satisfaction
14	EnvironmentSatisfaction	Satisfaction
15	WorkLifeBalance	Satisfaction
16	YearsSinceLastPromotion	Career growth
17	TrainingTimesLastYear	Training

```

>

```

Code description of task16: A `data.frame()` is created to organize the chosen analysis variables.

Title of the task17: Categorical to Numeric Conversion

Description of the task17: This task creates numeric versions of selected categorical variables for numerical analysis.

Code for task17

```

hr$AttritionNum <- ifelse(hr$Attrition == "Yes", 1, 0)
hr$OverTimeNum <- ifelse(hr$OverTime == "Yes", 1, 0)

```

```
head(hr[, c("Attrition","AttritionNum","OverTime","OverTimeNum")])
table(hr$AttritionNum)
```

Sample output of your code for task17

```
> hr$OverTimeNum <- ifelse(hr$OverTime == "Yes", 1, 0)
> head(hr[, c("Attrition","AttritionNum","OverTime","OverTimeNum")])
  Attrition AttritionNum OverTime OverTimeNum
1      Yes           1      Yes           1
2      No            0      No            0
3      Yes           1      Yes           1
4      No            0      Yes           1
5      No            0      No            0
6      No            0      No            0
> table(hr$AttritionNum)

  0    1 
1233 237 
> |
```

Code description of task17: The `ifelse()` function converts Yes/No categories into 1/0 form.

Title of the task18: Invalid Data Detection (Negative Values + Valid Range Checks)

Description of the task18: This task checks for negative values in numeric columns and verifies valid ranges for selected ordinal variables.

Code for task18

```
num_cols2 <- names(hr)[sapply(hr, is.numeric)]

neg_tbl <- sapply(hr[num_cols2], function(x) sum(x < 0, na.rm = TRUE))
neg_tbl[neg_tbl > 0]
sum(neg_tbl)

range_rules <- list(
  Education = c(1,5),
  EnvironmentSatisfaction = c(1,4),
  JobSatisfaction = c(1,4),
  WorkLifeBalance = c(1,4),
```

```

RelationshipSatisfaction = c(1,4),
JobInvolvement = c(1,4),
PerformanceRating = c(1,4),
StockOptionLevel = c(0,3)
)

range_check <- data.frame(
  Variable = names(range_rules),
  MinAllowed = sapply(range_rules, function(v) v[1]),
  MaxAllowed = sapply(range_rules, function(v) v[2]),
  InvalidCount = sapply(names(range_rules), function(v) {
    sum(hr[[v]] < range_rules[[v]][1] | hr[[v]] > range_rules[[v]][2], na.rm = TRUE)
  }),
  stringsAsFactors = FALSE
)

```

range_check

Sample output of your code for task18

```

+ )
> range_check

```

	Variable	MinAllowed	MaxAllowed	InvalidCount
Education	Education	1	5	0
EnvironmentSatisfaction	EnvironmentSatisfaction	1	4	0
JobSatisfaction	JobSatisfaction	1	4	0
WorkLifeBalance	WorkLifeBalance	1	4	0
RelationshipSatisfaction	RelationshipSatisfaction	1	4	0
JobInvolvement	JobInvolvement	1	4	0
PerformanceRating	PerformanceRating	1	4	0
StockOptionLevel	StockOptionLevel	0	3	0

```

> |

```

Code description of task18: Logical conditions are used to identify invalid values and count them.

Title of the task19: Filtering Example (OverTime = Yes and Age > 40)

Description of the task19: Create a subset to analyze a specific employee group and preview it.

Code for task19

```
subset_ot_age <- subset(hr, OverTime == "Yes" & Age > 40)
nrow(subset_ot_age)

head(subset_ot_age[,
c("Age", "Attrition", "Department", "JobRole", "MonthlyIncome", "OverTime")])
```

Sample output of your code for task19

```
> nrow(subset_ot_age)
[1] 144
> head(subset_ot_age[, c("Age", "Attrition", "Department", "JobRole", "MonthlyIncome", "OverTime")])
  Age Attrition Department JobRole MonthlyIncome OverTime
1  41      Yes      Sales Sales Executive      5993      Yes
7  59     No Research & Development Laboratory Technician      2670      Yes
32 44     No Research & Development Healthcare Representative      6465      Yes
37 50      Yes      Sales Sales Representative      2683      Yes
49 46     No      Sales Sales Executive      5772      Yes
51 48      Yes Research & Development Laboratory Technician      5381      Yes
> |
```

Code description of task19: subset() filters rows based on logical conditions.

Title of the task20: Filtering Example (Department = Sales) and Attrition Rate

Description of the task20: Filter Sales employees and compute attrition distribution within Sales.

Code for task20

```
sales_df <- subset(hr, Department == "Sales")
sales_tab <- table(sales_df$Attrition)
sales_tab
round(prop.table(sales_tab) * 100, 2)
```

Sample output of your code for task20

```
> sales_tab <- table(sales_df$Attrition)
> sales_tab

No Yes
354 92
> round(prop.table(sales_tab) * 100, 2)

No Yes
79.37 20.63
> |
```

Code description of task20: Uses class proportions to compare attrition within a subpopulation.

Title of the task21: Outlier Detection Using IQR Rule

Description of the task21: Compute Q1, Q3, IQR bounds and outlier counts for selected numeric variables using the $1.5 \times \text{IQR}$ rule.

Code for task21

```
iqr_info <- function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  lower <- q1 - 1.5 * iqr
  upper <- q3 + 1.5 * iqr
  out <- sum(x < lower | x > upper, na.rm = TRUE)
  c(Q1 = q1, Q3 = q3, IQR = iqr, Lower = lower, Upper = upper, Outliers = out)
}

out_vars <- c("MonthlyIncome", "TotalWorkingYears")
outlier_table <- t(sapply(hr[out_vars], iqr_info))
round(outlier_table, 2)
```

Sample output of your code for task21

```
> outlier_table <- t(sapply(hr[out_vars], iqr_info))
> round(outlier_table, 2)
      Q1.25% Q3.75% IQR.75% Lower.25% Upper.75% Outliers
MonthlyIncome    2911    8379    5468    -5291.0    16581.0    114
TotalWorkingYears      6     15      9      -7.5     28.5     63
> |
```

Code description of task21: Quartiles and IQR are calculated, and values beyond the IQR bounds are counted as outliers.

Title of the task22: Outlier Handling by Winsorization (Capping)

Description of the task22: Cap values below/above IQR bounds instead of deleting them (winsorization).

Code for task22

```
cap_iqr <- function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  lower <- q1 - 1.5 * iqr
  upper <- q3 + 1.5 * iqr
  x2 <- x
  x2[x2 < lower] <- lower
  x2[x2 > upper] <- upper
  list(Capped = x2, Lower = lower, Upper = upper, Changed = sum(x2 != x, na.rm = TRUE))
}
```

```
income_cap <- cap_iqr(hr$MonthlyIncome)
hr$MonthlyIncome_capped <- income_cap$Capped
```

```
twy_cap <- cap_iqr(hr$TotalWorkingYears)
hr$TotalWorkingYears_capped <- twy_cap$Capped
```

```
c(Lower = income_cap$Lower, Upper = income_cap$Upper, Changed =
income_cap$Changed)
```

```
summary(hr$MonthlyIncome)
summary(hr$MonthlyIncome_capped)
```

```
c(Lower = twy_cap$Lower, Upper = twy_cap$Upper, Changed = twy_cap$Changed)
summary(hr$TotalWorkingYears)
summary(hr$TotalWorkingYears_capped)
```

Sample output of your code for task22

```
> c(Lower = income_cap$Lower, Upper = income_cap$Upper, Changed = income_cap$Changed)
Lower.25% Upper.75%   Changed
    -5291    16581      114
> summary(hr$MonthlyIncome)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1009   2911   4919   6503   8379  19999
> summary(hr$MonthlyIncome_capped)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1009   2911   4919   6362   8379  16581
>
> c(Lower = twy_cap$Lower, Upper = twy_cap$Upper, Changed = twy_cap$Changed)
Lower.25% Upper.75%   Changed
    -7.5    28.5    63.0
> summary(hr$TotalWorkingYears)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.00   6.00  10.00  11.28  15.00  40.00
> summary(hr$TotalWorkingYears_capped)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.0   6.0   10.0   11.1  15.0   28.5
>
```

Code description of task22: Winsorization reduces extreme influence while preserving all rows.

Title of the task23: Boxplot Before vs After Outlier Treatment

Description of the task23: This task visually compares the selected variable before and after outlier treatment.

Code for task23

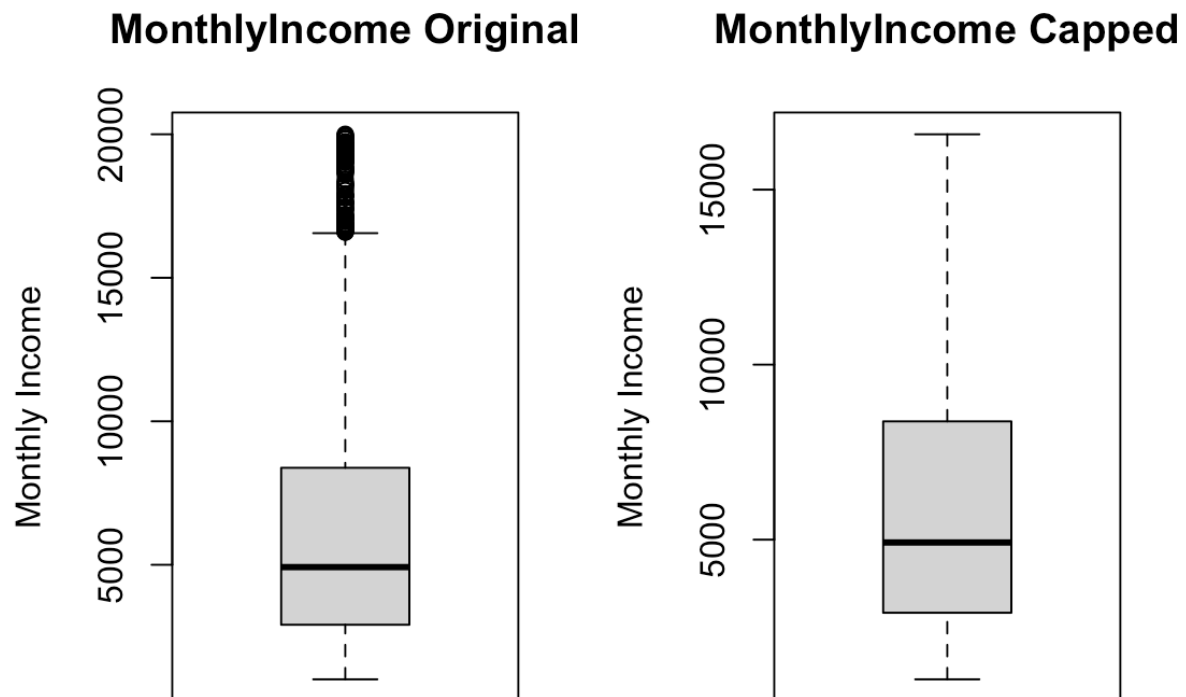
```
par(mfrow = c(1,2))

boxplot(hr$MonthlyIncome, main = "MonthlyIncome Original", ylab = "Monthly Income")

boxplot(hr$MonthlyIncome_capped, main = "MonthlyIncome Capped", ylab = "Monthly Income")

par(mfrow = c(1,1))
```

Sample output of your code for task23



Code description of task23: Two boxplots are displayed side by side using `par(mfrow=...)`.

Title of the task24: Min-Max Normalization

Description of the task24: Normalize continuous variables to the [0,1] range using min-max scaling.

Code for task24

```
minmax <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}
hr$MonthlyIncome_norm <- minmax(hr$MonthlyIncome_capped)
hr$DistanceFromHome_norm <- minmax(hr$DistanceFromHome)
```

```
head(hr[,
c("MonthlyIncome_capped","MonthlyIncome_norm","DistanceFromHome","DistanceFrom
Home_norm"))

summary(hr$MonthlyIncome_norm)
```

Sample output of your code for task24

```
> head(hr[, c("MonthlyIncome_capped","MonthlyIncome_norm","DistanceFromHome","DistanceFromHome_norm")])
  MonthlyIncome_capped MonthlyIncome_norm DistanceFromHome DistanceFromHome_norm
1                5993          0.32006165                1          0.00000000
2                 5130          0.26464166                8          0.25000000
3                 2090          0.06941947                2          0.03571429
4                 2909          0.12201387                3          0.07142857
5                 3468          0.15791164                2          0.03571429
6                 3068          0.13222451                2          0.03571429
> summary(hr$MonthlyIncome_norm)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.0000 0.1221  0.2511  0.3438  0.4733  1.0000
> |
```

Code description of task24: Scaling prevents large-scale attributes from dominating comparisons.

Title of the task25: Numeric to Categorical Conversion

Description of the task25: Convert numeric fields into categories (AgeGroup, IncomeGroup) for grouped analysis.

Code for task25

```
hr$AgeGroup <- cut(
  hr$Age,
  breaks = c(17,29,39,49,60),
  labels = c("18-29","30-39","40-49","50-60"),
  include.lowest = TRUE
)

hr$IncomeGroup <- cut(
  hr$MonthlyIncome_capped,
  breaks = quantile(hr$MonthlyIncome_capped, probs = c(0,0.33,0.66,1), na.rm = TRUE),
  labels = c("Low","Medium","High"),
  include.lowest = TRUE
)
```

```
table(hr$AgeGroup)
table(hr$IncomeGroup)
table(hr$AgeGroup, hr$Attrition)
```

Sample output of your code for task25

```
. /
>
> table(hr$AgeGroup)

18-29 30-39 40-49 50-60
  326   622   349   173
> table(hr$IncomeGroup)

  Low Medium   High
  485   485   500
> table(hr$AgeGroup, hr$Attrition)

      No Yes
18-29 235  91
30-39 533  89
40-49 315  34
50-60 150  23
>
```

Code description of task25: cut() bins continuous data into interpretable categories.

Title of the task26: Descriptive Statistics by Class (Attrition) for Age

Description of the task26: Compute mean, median, SD, variance, IQR, min, max, range for Age by Attrition.

Code for task26

```
calc_stats <- function(x) {
  c(
    Mean = mean(x, na.rm = TRUE),
    Median = median(x, na.rm = TRUE),
    SD = sd(x, na.rm = TRUE),
    Variance = var(x, na.rm = TRUE),
    IQR = IQR(x, na.rm = TRUE),
    Min = min(x, na.rm = TRUE),
    Max = max(x, na.rm = TRUE),
    Range = max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
  )
}
```

```
}
```

```
age_by_attr <- t(sapply(split(hr$Age, hr$Attrition), calc_stats))
round(age_by_attr, 2)
```

Sample output of your code for task26

```
> age_by_attr <- t(sapply(split(hr$Age, hr$Attrition), calc_stats))
> round(age_by_attr, 2)
      Mean Median   SD Variance IQR Min Max Range
No  37.56    36 8.89   79.00  12  18  60   42
Yes 33.61    32 9.69   93.88  11  18  58   40
>
```

Code description of task26: split() separates values by class; sapply() computes stats per class.

Title of the task27: Descriptive Statistics by Class (Attrition) for Monthly Income

Description of the task27: Compute the same statistics for MonthlyIncome (capped) by Attrition.

Code for task27

```
income_by_attr <- t(sapply(split(hr$MonthlyIncome_capped, hr$Attrition), calc_stats))
round(income_by_attr, 2)
```

Sample output of your code for task27

```
      Mean Median   SD Variance IQR Min  Max Range
No  6676.28   5204 4444.82 19756448 5623 1051 16581 15530
Yes 4726.27   3202 3409.90 11627450 3543 1009 16581 15572
>
```

Code description of task27: Same method as Task 26, applied to income.

Title of the task28: Mean Comparison (Required) — Monthly Income Across Attrition

Description of the task28: Compare average MonthlyIncome between Attrition groups using aggregate().

Code for task28

```
mean_income <- aggregate(MonthlyIncome_capped ~ Attrition, data = hr, FUN = mean)
mean_income
```

Sample output of your code for task28

```
> mean_income
  Attrition MonthlyIncome_capped
1       No           6676.281
2       Yes           4726.274
> |
```

Code description of task28: aggregate() computes summary statistics by category.

Title of the task29: Mean Comparison — Total Working Years Across OverTime

Description of the task29: Compare mean TotalWorkingYears between OverTime categories (Yes/No).

Code for task29

```
aggregate(TotalWorkingYears ~ OverTime, data = hr, FUN = mean)
```

Sample output of your code for task29

```
> aggregate(TotalWorkingYears ~ OverTime, data = hr, FUN = mean)
  OverTime TotalWorkingYears
1       No           11.21727
2       Yes           11.43750
> |
```

Code description of task29: Another category-based mean comparison using aggregate().

Title of the task30: Variability Comparison (Required) — Total Working Years Across Attrition

Description of the task30: This task calculates the variability measures (Range, IQR, Variance, and Standard Deviation) of TotalWorkingYears for different Attrition groups.

Code for task30

```
spread_twy <- aggregate(
  TotalWorkingYears_capped ~ Attrition,
  data = hr,
  FUN = function(x) c(
    Range = max(x) - min(x),
    IQR = IQR(x),
    Variance = var(x),
    SD = sd(x)
  )
)

spread_twy_df <- data.frame(
  Attrition = spread_twy$Attrition,
  spread_twy$TotalWorkingYears_capped
)

names(spread_twy_df) <- c("Attrition", "Range", "IQR", "Variance", "SD")

spread_twy_df[, -1] <- round(spread_twy_df[, -1], 2)

spread_twy_df
```

Sample output of your code for task30

```
> spread_twy_df
  Attrition Range IQR Variance  SD
1      No  28.5  10    53.12  7.29
2     Yes  28.5   7    43.92  6.63
>
```

Code description of task30: The aggregate() function is used to compute variability measures for each Attrition category. The round() function is applied only to numeric columns to format the output.

Title of the task31: Variability Across Multiple Levels — Monthly Income by Department

Description of the task31: This task calculates the variability measures (Range, IQR, Variance, and Standard Deviation) of MonthlyIncome across different departments.

Code for task31

```
spread_income_dept <- aggregate(
  MonthlyIncome_capped ~ Department,
  data = hr,
  FUN = function(x) c(
    Range = max(x) - min(x),
    IQR = IQR(x),
    Variance = var(x),
    SD = sd(x)
  )
)

spread_income_df <- data.frame(
  Department = spread_income_dept$Department,
  spread_income_dept$MonthlyIncome_capped
)

names(spread_income_df) <- c("Department", "Range", "IQR", "Variance", "SD")

spread_income_df[, -1] <- round(spread_income_df[, -1], 2)
```

spread_income_df

Sample output of your code for task31

```
>
> spread_income_df
      Department Range    IQR Variance    SD
1      Human Resources 15026 5768.5 26427722 5140.79
2 Research & Development 15572 4914.0 20370795 4513.40
3              Sales 15529 4330.5 14554339 3815.01
> |
```

Code description of task31: The aggregate() function is used to compute variability measures for each department. The round() function is applied only to numeric columns to avoid errors from categorical variables.

Title of the task32: Correlation Matrix for Selected Numeric Variables

Description of the task32: Compute correlations among selected numeric variables and print the matrix.

Code for task32

```
cor_vars <- c("Age", "MonthlyIncome_capped", "TotalWorkingYears_capped",
             "DistanceFromHome", "YearsAtCompany", "JobLevel", "PercentSalaryHike")

cor_mat <- cor(hr[, cor_vars])

round(cor_mat, 3)
```

Sample output of your code for task32

```
> cor_mat <- cor(hr[, cor_vars])
> round(cor_mat, 3)
      Age MonthlyIncome_capped TotalWorkingYears_capped DistanceFromHome YearsAtCompany
Age      1.000              0.500              0.678             -0.002              0.311
MonthlyIncome_capped 0.500              1.000              0.776             -0.009              0.514
TotalWorkingYears_capped 0.678              0.776              1.000              0.005              0.620
DistanceFromHome    -0.002             -0.009              0.005              1.000              0.010
YearsAtCompany       0.311              0.514              0.620              0.010              1.000
JobLevel             0.510              0.946              0.787              0.005              0.535
PercentSalaryHike    0.004             -0.025             -0.022              0.040             -0.036
      JobLevel PercentSalaryHike
Age      0.510              0.004
MonthlyIncome_capped 0.946             -0.025
TotalWorkingYears_capped 0.787             -0.022
DistanceFromHome      0.005              0.040
YearsAtCompany        0.535             -0.036
JobLevel              1.000             -0.035
PercentSalaryHike    -0.035              1.000
> |
```

Code description of task32: cor() measures linear relationships among numeric columns.

Title of the task33: Welch Two-Sample t-test (Income vs Attrition)

Description of the task33: Test whether mean income differs between Attrition groups; include as an extra analysis.

Code for task33

```
t.test(MonthlyIncome_capped ~ Attrition, data = hr)
```

Sample output of your code for task33

```
> t.test(MonthlyIncome_capped ~ Attrition, data = hr)

Welch Two Sample t-test

data: MonthlyIncome_capped by Attrition
t = 7.6436, df = 407.01, p-value = 1.533e-13
alternative hypothesis: true difference in means between group No and group Yes is not equal to 0
95 percent confidence interval:
 1448.497 2451.515
sample estimates:
mean in group No mean in group Yes
    6676.281      4726.274

> |
```

Code description of task33: t.test() performs a two-sample test; treat results as statistical evidence, while noting it does not prove causation.

Title of the task34: Histograms (Age and Monthly Income)

Description of the task34: Plot distributions for two important numeric variables.

Code for task34

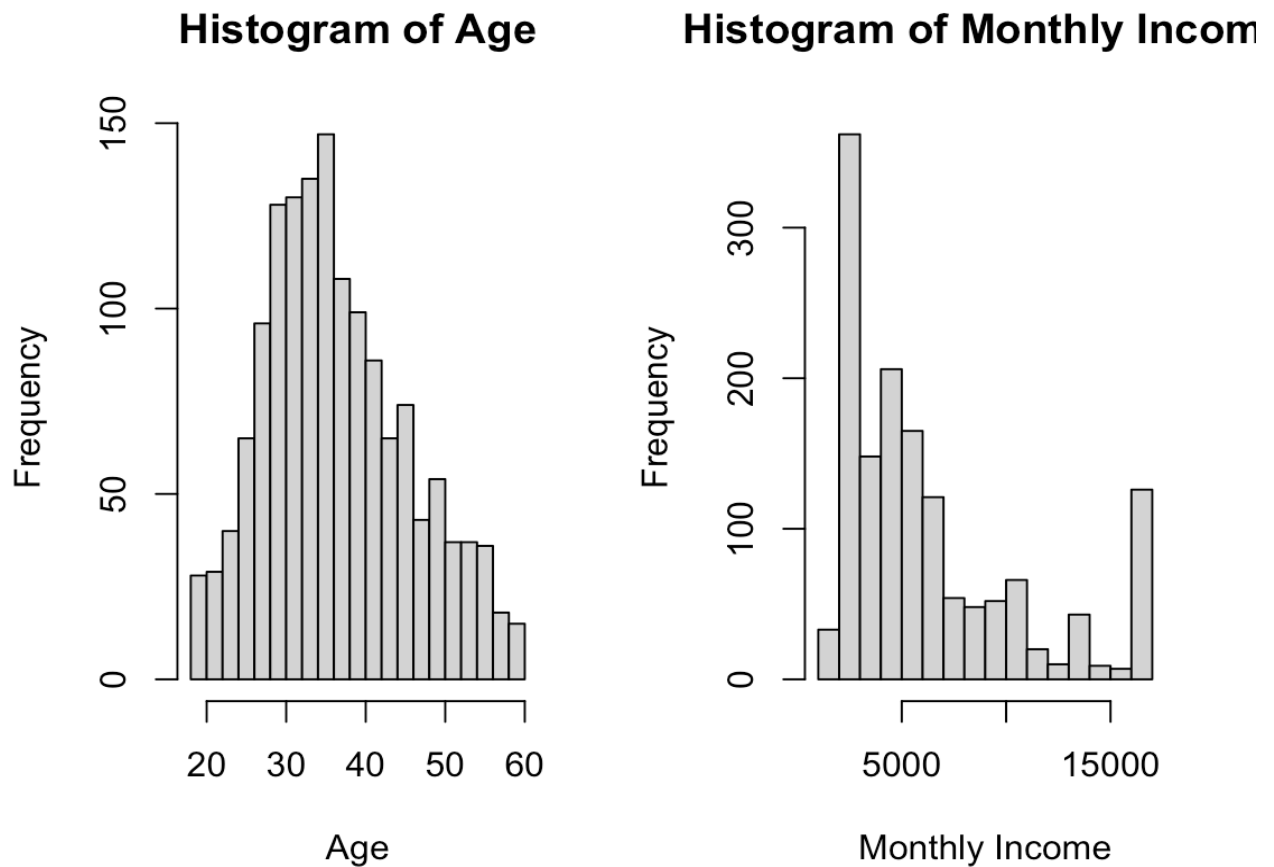
```
par(mfrow = c(1,2))

hist(hr$Age, breaks = 15, main = "Histogram of Age", xlab = "Age")

hist(hr$MonthlyIncome_capped, breaks = 20, main = "Histogram of Monthly Income", xlab = "Monthly Income")

par(mfrow = c(1,1))
```

Sample output of your code for task34



Code description of task34: Histograms summarize distribution shapes and frequency.

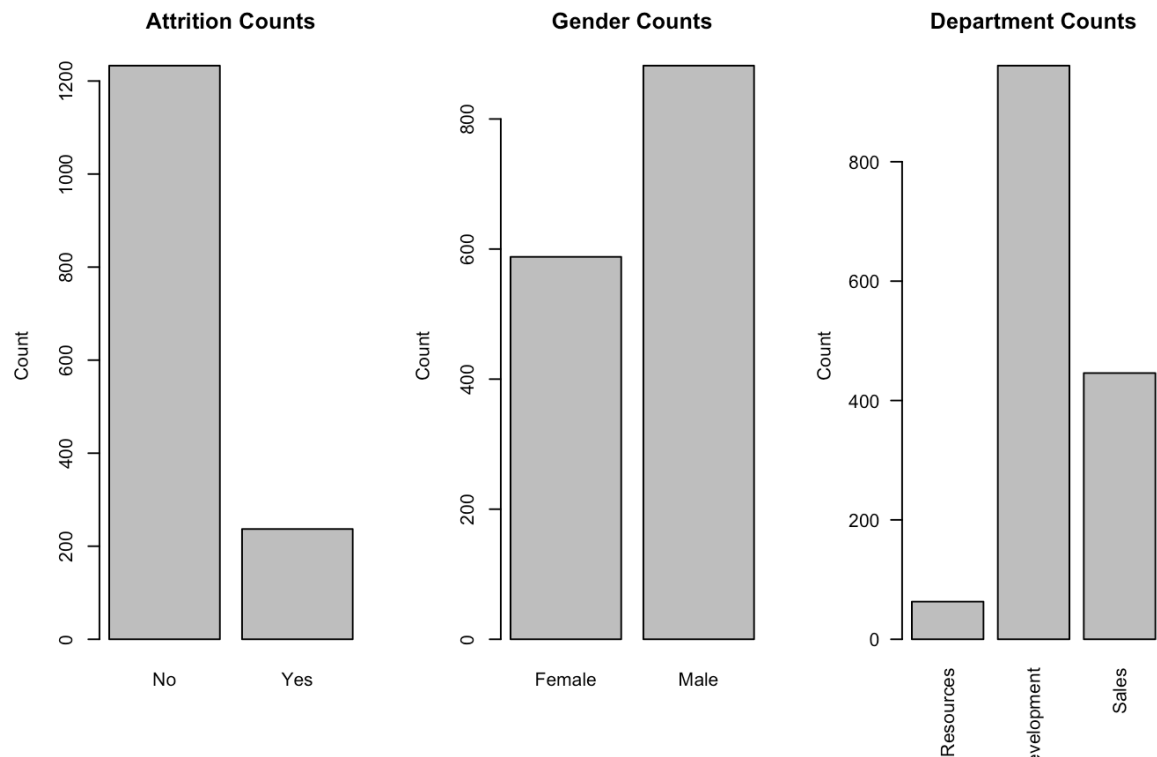
Title of the task35: Bar Charts (Attrition, Gender, Department)

Description of the task35: Visualize categorical distributions with bar charts.

Code for task35

```
par(mfrow = c(1,3))
barplot(table(hr$Attrition), main = "Attrition Counts", ylab = "Count")
barplot(table(hr$Gender), main = "Gender Counts", ylab = "Count")
barplot(table(hr$Department), main = "Department Counts", ylab = "Count", las = 2)
par(mfrow = c(1,1))
```

Sample output of your code for task35



Code description of task35: `barplot()` visualizes category counts

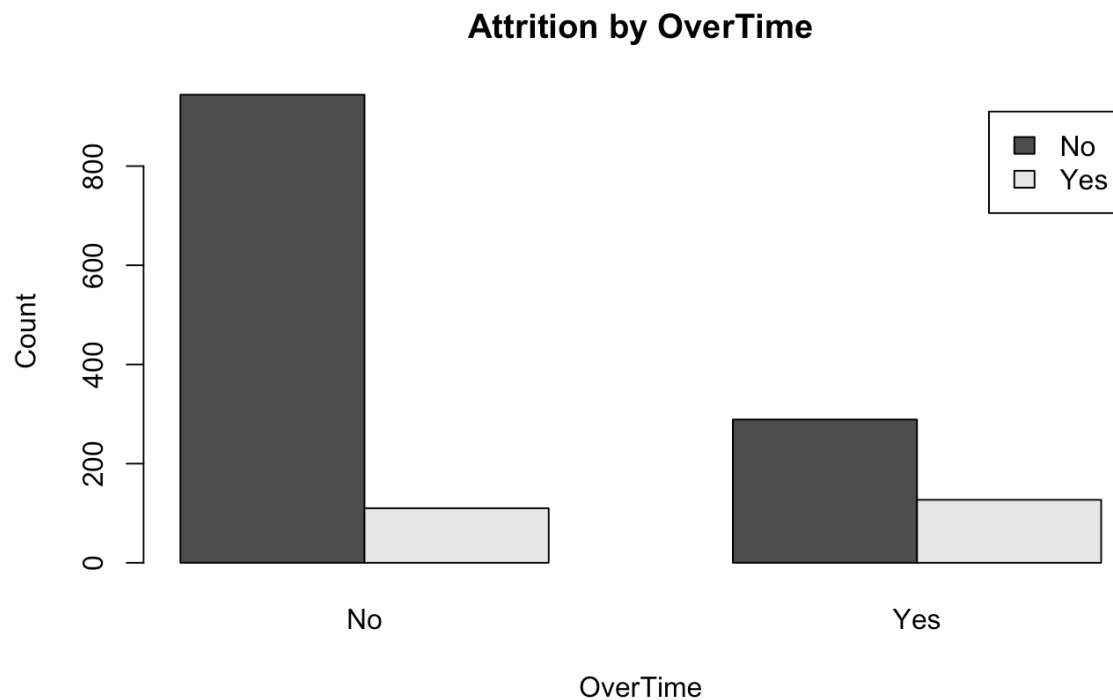
Title of the task36: Attrition by OverTime (Cross-tab + Grouped Barplot)

Description of the task36: Compare Attrition counts across OverTime categories.

Code for task36

```
ot_tab <- table(hr$OverTime, hr$Attrition)
ot_tab
barplot(t(ot_tab), beside = TRUE, legend.text = TRUE,
        main = "Attrition by OverTime", xlab = "OverTime", ylab = "Count")
```

Sample output of your code for task36



Code description of task36: `table()` creates a contingency table; `barplot()` displays it visually.

Title of the task37: Boxplot by Class + Scatter Plot

Description of the task37: Visualize income differences by Attrition and relationship between Age and Income.

Code for task37

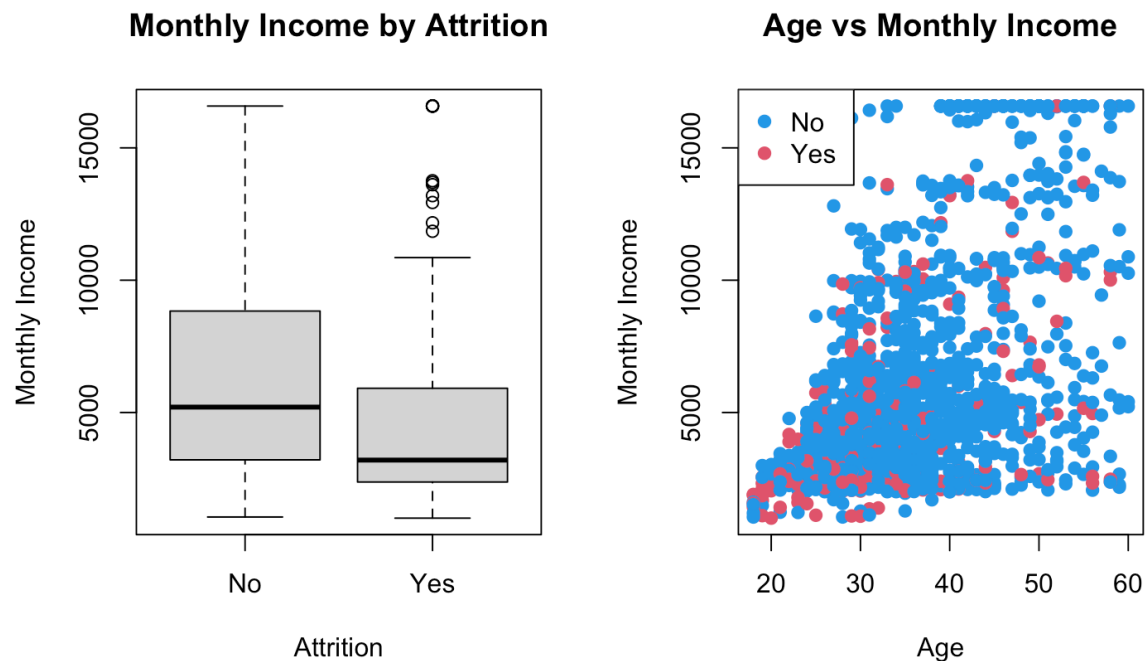
```
par(mfrow = c(1,2))
boxplot(MonthlyIncome_capped ~ Attrition, data = hr,
        main = "Monthly Income by Attrition",
        xlab = "Attrition", ylab = "Monthly Income")

plot(hr$Age, hr$MonthlyIncome_capped, pch = 19,
     col = ifelse(hr$Attrition == "Yes", 2, 4),
     main = "Age vs Monthly Income",
     xlab = "Age", ylab = "Monthly Income")
```

```
legend("topleft", legend = c("No","Yes"), col = c(4,2), pch = 19)
```

```
par(mfrow = c(1,1))
```

Sample output of your code for task37



Code description of task37: Boxplot compares distributions by category; scatter shows relationship and class separation.

Title of the task38: Correlation Heatmap (Selected Variables)

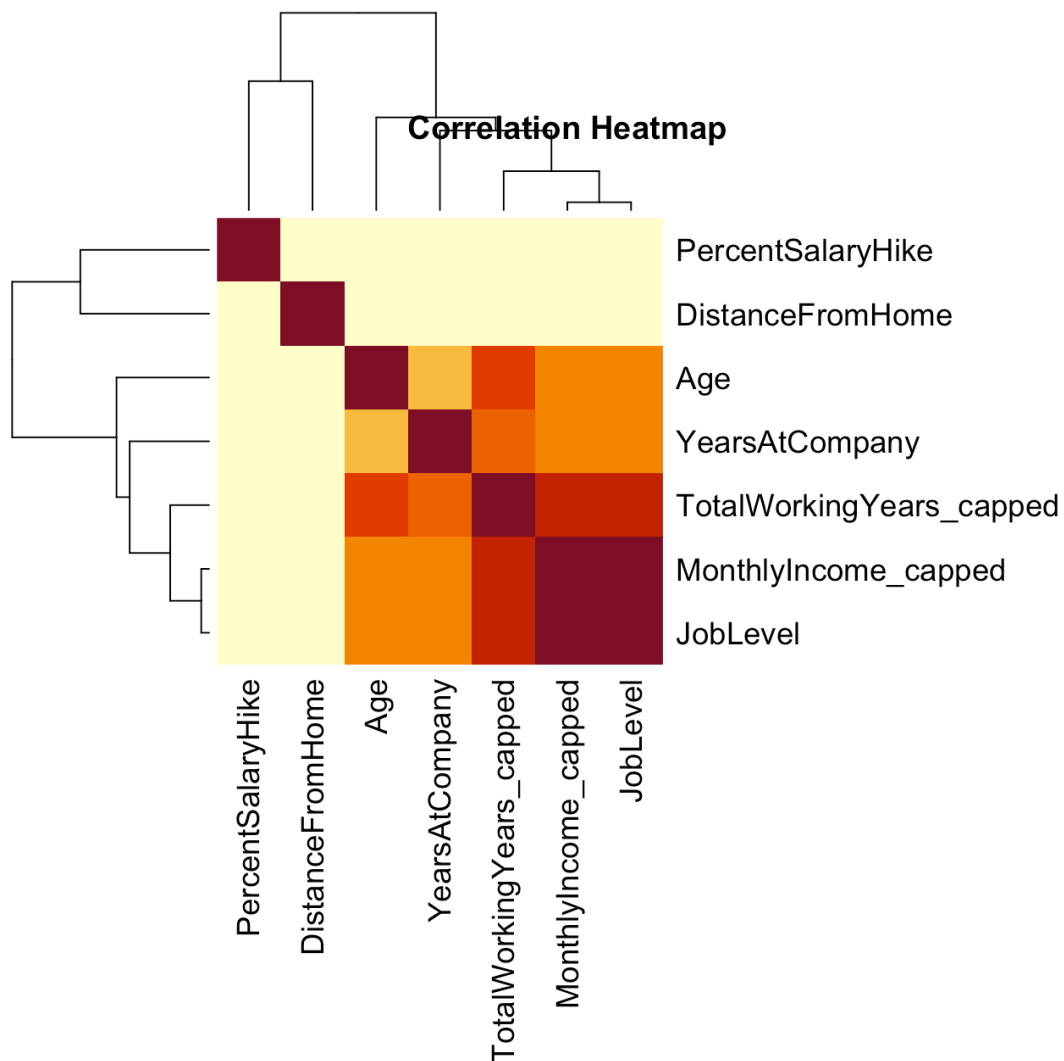
Description of the task38: Visualize the correlation matrix in heatmap form.

Code for task38

```
heatmap(cor_mat, symm = TRUE, margins = c(15,15))
```

```
title("Correlation Heatmap")
```

Sample output of your code for task38



Code description of task38: heatmap() produces a color-based correlation visualization.

Title of the task39: Stratified Train-Test Split (70%/30%)

Description of the task39: Split dataset into training and testing sets while preserving Attrition ratio.

Code for task39

```
set.seed(42)

train_idx <- unlist(tapply(1:nrow(hr), hr$Attrition, function(idx) {
  sample(idx, size = round(0.7 * length(idx)))
}))

train <- hr[train_idx, ]
test <- hr[-train_idx, ]
```

```
c(TrainRows = nrow(train), TestRows = nrow(test))
```

```
table(train$Attrition)
```

```
table(test$Attrition)
```

Sample output of your code for task39

```
> c(TrainRows = nrow(train), TestRows = nrow(test))
TrainRows TestRows
      1029      441
> table(train$Attrition)

No Yes
863 166
> table(test$Attrition)

No Yes
370  71
>
```

Code description of task39: The `tapply()` function is used to group indices by Attrition class, and `sample()` selects 70% of data from each class. This ensures balanced class distribution in both training and testing sets.

Title of the task40: Class Imbalance Handling (Simple Oversampling for Training Set)

Description of the task40: This task balances the training dataset by applying oversampling to the minority class (Attrition = Yes). This ensures equal representation of both classes.

Code for task40

```
majority_train <- train[train$Attrition == "No", ]
minority_train <- train[train$Attrition == "Yes", ]
set.seed(42)

minority_over <- minority_train[sample(1:nrow(minority_train), nrow(majority_train),
replace = TRUE), ]

train_balanced <- rbind(majority_train, minority_over)
table(train_balanced$Attrition)

nrow(train_balanced)
```

Sample output of your code for task40

```
> table(train_balanced$Attrition)

No Yes
863 863
> nrow(train_balanced)
[1] 1726
> |
```

Code description of task40: The minority class is randomly sampled with replacement using the `sample()` function until it matches the size of the majority class. The datasets are then combined using `rbind()`.

Project Code

```

hr_raw <- read.csv("WA_Fn-UseC_-HR-Employee-Attrition.csv", stringsAsFactors =
FALSE)

head(hr_raw[, c("Age","Attrition","Department","JobRole","MonthlyIncome","OverTime")])

dim(hr_raw)
names(hr_raw)
str(hr_raw)

numeric_cols <- names(hr_raw)[sapply(hr_raw, is.numeric)]
categorical_cols <- names(hr_raw)[sapply(hr_raw, is.character)]

length(numeric_cols)
length(categorical_cols)

numeric_cols
categorical_cols

ds_summary <- data.frame(
  Dataset = "IBM HR Analytics Employee Attrition",
  Rows = nrow(hr_raw),
  Columns = ncol(hr_raw),
  Numeric_Columns = length(numeric_cols),
  Categorical_Columns = length(categorical_cols),
  Target = "Attrition (Yes/No)",
  stringsAsFactors = FALSE
)
ds_summary

attr_table <- table(hr_raw$Attrition)

```

```
attr_percent <- round(prop.table(attr_table) * 100, 2)
```

```
attr_table
```

```
attr_percent
```

```
summary(hr_raw[, c("Age", "MonthlyIncome", "TotalWorkingYears", "DistanceFromHome")])
```

```
na_col <- colSums(is.na(hr_raw))
```

```
na_col
```

```
sum(na_col)
```

```
hr_step <- hr_raw
```

```
for (col in categorical_cols) {
```

```
  hr_step[[col]][trimws(hr_step[[col]]) == ""] <- NA
```

```
}
```

```
na_col2 <- colSums(is.na(hr_step))
```

```
na_col2
```

```
sum(na_col2)
```

```
get_mode <- function(x) {
```

```
  ux <- unique(x[!is.na(x)])
```

```
  ux[which.max(tabulate(match(x, ux)))]
```

```
}
```

```
hr_imputed <- hr_step
```

```
na_before <- colSums(is.na(hr_imputed))
```

```
for (col in numeric_cols) {
```

```
  hr_imputed[[col]][is.na(hr_imputed[[col]])] <- median(hr_imputed[[col]], na.rm = TRUE)
```

```
}
```

```

for (col in categorical_cols) {
  hr_imputed[[col]][is.na(hr_imputed[[col]])] <- get_mode(hr_imputed[[col]])
}

na_before
na_after <- colSums(is.na(hr_imputed))
na_after

remove_tbl <- data.frame(
  Column = c("EmployeeCount", "EmployeeNumber", "Over18", "StandardHours"),
  Reason = c("Constant column", "Identifier-like column", "Constant column", "Constant
column"),
  UniqueValues = c(
    length(unique(hr_imputed$EmployeeCount)),
    length(unique(hr_imputed$EmployeeNumber)),
    length(unique(hr_imputed$Over18)),
    length(unique(hr_imputed$StandardHours))
  ),
  stringsAsFactors = FALSE
)
remove_tbl

remove_cols <- c("EmployeeCount", "EmployeeNumber", "Over18", "StandardHours")
hr <- hr_imputed[, !(names(hr_imputed) %in% remove_cols)]
dim(hr)
setdiff(remove_cols, names(hr))

dup_before <- sum(duplicated(hr))
rows_before <- nrow(hr)
hr <- hr[!duplicated(hr), ]
rows_after <- nrow(hr)

```

```

dup_before
c(Before = rows_before, After = rows_after)

char_cols <- names(hr)[sapply(hr, is.character)]
for (col in char_cols) {
  hr[[col]] <- as.factor(hr[[col]])
}
char_cols
sapply(hr[char_cols], class)
lapply(hr[, c("Attrition","OverTime","Department")], levels)

analysis_vars <- data.frame(
  Variable = c("Attrition","Age","Gender","Department","JobRole","OverTime",
    "MonthlyIncome","DailyRate","DistanceFromHome","TotalWorkingYears",
    "YearsAtCompany","JobLevel","JobSatisfaction","EnvironmentSatisfaction",
    "WorkLifeBalance","YearsSinceLastPromotion","TrainingTimesLastYear"),
  Role = c("Target","Demographic","Demographic","Work context","Work
context","Workload",
    "Compensation","Compensation","Commute","Experience",
    "Tenure","Seniority","Satisfaction","Satisfaction",
    "Satisfaction","Career growth","Training"),
  stringsAsFactors = FALSE
)
analysis_vars

hr$AttritionNum <- ifelse(hr$Attrition == "Yes", 1, 0)
hr$OverTimeNum <- ifelse(hr$OverTime == "Yes", 1, 0)
head(hr[, c("Attrition","AttritionNum","OverTime","OverTimeNum")])
table(hr$AttritionNum)

num_cols2 <- names(hr)[sapply(hr, is.numeric)]

```

```
neg_tbl <- sapply(hr[num_cols2], function(x) sum(x < 0, na.rm = TRUE))
neg_tbl[neg_tbl > 0]
sum(neg_tbl)
```

```
range_rules <- list(
  Education = c(1,5),
  EnvironmentSatisfaction = c(1,4),
  JobSatisfaction = c(1,4),
  WorkLifeBalance = c(1,4),
  RelationshipSatisfaction = c(1,4),
  JobInvolvement = c(1,4),
  PerformanceRating = c(1,4),
  StockOptionLevel = c(0,3)
)
```

```
range_check <- data.frame(
  Variable = names(range_rules),
  MinAllowed = sapply(range_rules, function(v) v[1]),
  MaxAllowed = sapply(range_rules, function(v) v[2]),
  InvalidCount = sapply(names(range_rules), function(v) {
    sum(hr[[v]] < range_rules[[v]][1] | hr[[v]] > range_rules[[v]][2], na.rm = TRUE)
  }),
  stringsAsFactors = FALSE
)
range_check
```

```
subset_ot_age <- subset(hr, OverTime == "Yes" & Age > 40)
nrow(subset_ot_age)
head(subset_ot_age[,
  c("Age", "Attrition", "Department", "JobRole", "MonthlyIncome", "OverTime")])
```

```
sales_df <- subset(hr, Department == "Sales")
```

```
sales_tab <- table(sales_df$Attrition)
```

```
sales_tab
```

```
round(prop.table(sales_tab) * 100, 2)
```

```
iqr_info <- function(x) {
```

```
  q1 <- quantile(x, 0.25, na.rm = TRUE)
```

```
  q3 <- quantile(x, 0.75, na.rm = TRUE)
```

```
  iqr <- q3 - q1
```

```
  lower <- q1 - 1.5 * iqr
```

```
  upper <- q3 + 1.5 * iqr
```

```
  out <- sum(x < lower | x > upper, na.rm = TRUE)
```

```
  c(Q1 = q1, Q3 = q3, IQR = iqr, Lower = lower, Upper = upper, Outliers = out)
```

```
}
```

```
out_vars <- c("MonthlyIncome", "TotalWorkingYears")
```

```
outlier_table <- t(sapply(hr[out_vars], iqr_info))
```

```
round(outlier_table, 2)
```

```
cap_iqr <- function(x) {
```

```
  q1 <- quantile(x, 0.25, na.rm = TRUE)
```

```
  q3 <- quantile(x, 0.75, na.rm = TRUE)
```

```
  iqr <- q3 - q1
```

```
  lower <- q1 - 1.5 * iqr
```

```
  upper <- q3 + 1.5 * iqr
```

```
  x2 <- x
```

```
  x2[x2 < lower] <- lower
```

```
  x2[x2 > upper] <- upper
```

```
  list(Capped = x2, Lower = lower, Upper = upper, Changed = sum(x2 != x, na.rm = TRUE))
```

```
}
```

```

income_cap <- cap_iqr(hr$MonthlyIncome)
hr$MonthlyIncome_capped <- income_cap$Capped

twy_cap <- cap_iqr(hr$TotalWorkingYears)
hr$TotalWorkingYears_capped <- twy_cap$Capped

c(Lower = income_cap$Lower, Upper = income_cap$Upper, Changed =
income_cap$Changed)
summary(hr$MonthlyIncome)
summary(hr$MonthlyIncome_capped)

c(Lower = twy_cap$Lower, Upper = twy_cap$Upper, Changed = twy_cap$Changed)
summary(hr$TotalWorkingYears)
summary(hr$TotalWorkingYears_capped)

par(mfrow = c(1,2))
boxplot(hr$MonthlyIncome, main = "MonthlyIncome Original", ylab = "Monthly Income")
boxplot(hr$MonthlyIncome_capped, main = "MonthlyIncome Capped", ylab = "Monthly
Income")
par(mfrow = c(1,1))

minmax <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

hr$MonthlyIncome_norm <- minmax(hr$MonthlyIncome_capped)
hr$DistanceFromHome_norm <- minmax(hr$DistanceFromHome)

head(hr[,
c("MonthlyIncome_capped", "MonthlyIncome_norm", "DistanceFromHome", "DistanceFrom
Home_norm")])

```

```
summary(hr$MonthlyIncome_norm)
```

```
hr$AgeGroup <- cut(
  hr$Age,
  breaks = c(17,29,39,49,60),
  labels = c("18-29","30-39","40-49","50-60"),
  include.lowest = TRUE
)
```

```
hr$IncomeGroup <- cut(
  hr$MonthlyIncome_capped,
  breaks = quantile(hr$MonthlyIncome_capped, probs = c(0,0.33,0.66,1), na.rm = TRUE),
  labels = c("Low","Medium","High"),
  include.lowest = TRUE
)
```

```
table(hr$AgeGroup)
```

```
table(hr$IncomeGroup)
```

```
table(hr$AgeGroup, hr$Attrition)
```

```
calc_stats <- function(x) {
  c(
    Mean = mean(x, na.rm = TRUE),
    Median = median(x, na.rm = TRUE),
    SD = sd(x, na.rm = TRUE),
    Variance = var(x, na.rm = TRUE),
    IQR = IQR(x, na.rm = TRUE),
    Min = min(x, na.rm = TRUE),
    Max = max(x, na.rm = TRUE),
    Range = max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
  )
}
```

```
)
}
```

```
age_by_attr <- t(sapply(split(hr$Age, hr$Attrition), calc_stats))
round(age_by_attr, 2)
```

```
income_by_attr <- t(sapply(split(hr$MonthlyIncome_capped, hr$Attrition), calc_stats))
round(income_by_attr, 2)
```

```
mean_income <- aggregate(MonthlyIncome_capped ~ Attrition, data = hr, FUN = mean)
mean_income
```

```
aggregate(TotalWorkingYears ~ OverTime, data = hr, FUN = mean)
```

```
spread_twy <- aggregate(
  TotalWorkingYears_capped ~ Attrition,
  data = hr,
  FUN = function(x) c(
    Range = max(x) - min(x),
    IQR = IQR(x),
    Variance = var(x),
    SD = sd(x)
  )
)
```

```
spread_twy_df <- data.frame(
  Attrition = spread_twy$Attrition,
  spread_twy$TotalWorkingYears_capped
)
```

```
names(spread_twy_df) <- c("Attrition","Range","IQR","Variance","SD")
```

```
spread_twy_df[, -1] <- round(spread_twy_df[, -1], 2)
```

```
spread_twy_df
```

```
spread_income_dept <- aggregate(
  MonthlyIncome_capped ~ Department,
  data = hr,
  FUN = function(x) c(
    Range = max(x) - min(x),
    IQR = IQR(x),
    Variance = var(x),
    SD = sd(x)
  )
)
```

```
spread_income_df <- data.frame(
  Department = spread_income_dept$Department,
  spread_income_dept$MonthlyIncome_capped
)
```

```
names(spread_income_df) <- c("Department","Range","IQR","Variance","SD")
```

```
spread_income_df[, -1] <- round(spread_income_df[, -1], 2)
```

```
spread_income_df
```

```

cor_vars <- c("Age", "MonthlyIncome_capped", "TotalWorkingYears_capped",
             "DistanceFromHome", "YearsAtCompany", "JobLevel", "PercentSalaryHike")
cor_mat <- cor(hr[, cor_vars])
round(cor_mat, 3)

t.test(MonthlyIncome_capped ~ Attrition, data = hr)

par(mfrow = c(1,2))
hist(hr$Age, breaks = 15, main = "Histogram of Age", xlab = "Age")
hist(hr$MonthlyIncome_capped, breaks = 20, main = "Histogram of Monthly Income", xlab = "Monthly Income")
par(mfrow = c(1,1))

par(mfrow = c(1,3))
barplot(table(hr$Attrition), main = "Attrition Counts", ylab = "Count")
barplot(table(hr$Gender), main = "Gender Counts", ylab = "Count")
barplot(table(hr$Department), main = "Department Counts", ylab = "Count", las = 2)
par(mfrow = c(1,1))

ot_tab <- table(hr$OverTime, hr$Attrition)
ot_tab
barplot(t(ot_tab), beside = TRUE, legend.text = TRUE,
       main = "Attrition by OverTime", xlab = "OverTime", ylab = "Count")

par(mfrow = c(1,2))
boxplot(MonthlyIncome_capped ~ Attrition, data = hr,
       main = "Monthly Income by Attrition",
       xlab = "Attrition", ylab = "Monthly Income")

plot(hr$Age, hr$MonthlyIncome_capped, pch = 19,
     col = ifelse(hr$Attrition == "Yes", 2, 4),

```

```

    main = "Age vs Monthly Income",
    xlab = "Age", ylab = "Monthly Income")

legend("topleft", legend = c("No","Yes"), col = c(4,2), pch = 19)
par(mfrow = c(1,1))

heatmap(cor_mat, symm = TRUE, margins = c(15,15))
title("Correlation Heatmap")

set.seed(42)
train_idx <- unlist(tapply(1:nrow(hr), hr$Attrition, function(idx) {
  sample(idx, size = round(0.7 * length(idx)))
}))

train <- hr[train_idx, ]
test <- hr[-train_idx, ]

c(TrainRows = nrow(train), TestRows = nrow(test))
table(train$Attrition)
table(test$Attrition)

majority_train <- train[train$Attrition == "No", ]
minority_train <- train[train$Attrition == "Yes", ]

set.seed(42)
minority_over <- minority_train[sample(1:nrow(minority_train), nrow(majority_train),
replace = TRUE), ]
train_balanced <- rbind(majority_train, minority_over)

table(train_balanced$Attrition)
nrow(train_balanced)

```

